

Module 03: Perception — Reading Sensor Data in Code

Learning Objectives

1. Write code that reads **sensor data** (keyboard input, mouse position, file data) and uses it to make decisions.
2. Understand that perception in code means **comparing expected input to actual input** (prediction error).
3. Filter noisy data to extract a clean signal.

Introduction

Perception is how an agent gathers information about the world. In coding, perception means reading data — from the keyboard, from a file, from a sensor, or from a web API. But raw data is messy! This module teaches you to read, clean, and interpret data: the coding version of perception.

Key Concepts

1. Input as Perception

Every `input()` statement, every `read_sensor()` call, every API request is an act of perception. Your code is asking: "What is happening in the world right now?"

```
temperature = read_sensor("thermometer") # Perception!
print(f"Current temperature: {temperature}°F")
```

2. Prediction Error in Code

What if the sensor sometimes returns garbage data (like -999°F on a sunny day)? That is a prediction error! Your code expected a reasonable number, but got something weird. Good code *handles* prediction errors:

```
temperature = read_sensor("thermometer")
if temperature < -50 or temperature > 150:
    print("⚠️ Sensor error detected! Using last known value.")
    temperature = last_good_reading
```

This is exactly what your brain does: it detects implausible inputs and falls back on its expectations (prior beliefs).

3. Filtering Noisy Data

Real sensor data is noisy. A temperature sensor might read 72, 73, 71, 74, 72, 500, 73. That "500" is an outlier. A **moving average** filter smooths the data:

```
readings = [72, 73, 71, 74, 72, 500, 73]
window_size = 3
smoothed = []
for i in range(len(readings) - window_size + 1):
    window = readings[i:i + window_size]
    smoothed.append(sum(window) / window_size)
```

Filtering = removing surprise that is probably noise, not signal.

Activities

Activity 1: Temperature Monitor

Write a program that reads temperature values from a list (simulating a sensor). Detect outliers (values more than 20° away from the average). Print a warning for each outlier. What percentage of readings were errors?

Activity 2: Mouse Tracker

Write a program that tracks the mouse cursor position 10 times per second. Add a "prediction" feature: after each reading, the program guesses where the mouse will be 0.1 seconds from now. Display the prediction error. Is the error larger when the mouse moves fast or slow?

Summary

Perception in code means reading and interpreting sensor data. Good code detects prediction errors (implausible inputs), handles noise through filtering, and falls back on expectations when data is unreliable. This is exactly how biological perception works through Active Inference.

References

- Downey, A. B. (2015). *Think Python*. Chapter 10.